CLONESCOPE

Phase 2 Learning Manual

Function & Block Representation

Restricted Type-1 Body-Level Clone Detection

Written for: IIT, University of Dhaka · SPL-1 · Raian Huq (BSSE1708)

Implementation language: C++17 | Source snapshot: uploaded CloneScope Phase 2 ZIP | Prepared: 28 October 2026

Purpose: A beginner-accessible, technically exact guide to the implementation you have built: how the functions work, why each important line is there, how to test it, which conclusions are justified, and how to defend it in a supervisor viva.

> **CORE PRINCIPLE**
>
> A Type-1 body-level candidate means selected function-body token types and lexemes match exactly. It is not a plagiarism verdict, not whole-program equivalence, and not proof that students copied work.

This manual was derived from the exact source and tests in the uploaded ZIP. It distinguishes verified implementation, observed local test results, and limitations. No AI/ML classifier or other unimplemented clone types are attributed to this phase.

Reading map

1. What works today — Inputs, outputs, finished implementation, and unimplemented features.

2. Architecture and C++ essentials — How modules cooperate and how to read C++ references, vectors, structs, streams and ranges.

3. Recap: preprocessing and tokenization — How the Phase 1 lexical foundation preserves positional evidence.

4. Brace regions and stack trace — Exact `CodeRegion` syntax, stack-like vector, nested braces, half-open bounds.

5. Function-body identification — Reverse parenthesis scan, `depth`, guards, examples, and heuristic failure modes.

6. Restricted Type-1 matching — Exact code for range validation, comparison, cross-product and match records.

7. Explainable report generation — Streaming, boundary guards, filename/line extraction and test assertions.

8. Application and folder analysis — `AnalyzedSourceFile`, input branches, all unique file pairs, reporting and scope.

9. Worked trace and regression — Real fixture tokens, line ranges, three dataset results and seven test commands.

10. Complexity and limitations — Precise cost model and important unsupported syntax.

11. Git and supervisor viva — Safe branch/PR process, oral questions with concise model answers.

12. Annotated source appendix — Exact copies of key Phase 2 source files with original line numbers.

> **HOW TO STUDY**
>
> Read chapters 1–3 once; then follow each algorithm with pencil and paper. For each source listing, predict the value of `i`, the vector contents, and the return value before reading the explanation. Run the actual fixture tests on your Windows machine.

1. What CloneScope actually does now

CloneScope is currently a C++17 command-line tool. With a directory path, it scans directly contained lowercase `.c`/`.cpp` files, preprocesses comments, tokenizes the source, recognizes selected function bodies, stores per-file analysis in memory, compares every distinct pair of files once, and prints any exact function-body token matches with filenames and line ranges.

With a single-file path, the current application reads, preprocesses, tokenizes, and counts recognized function bodies; it does not run cross-file comparisons.

Part	Input	Output / observed behavior
Comment remover	Source text	Text with supported comments replaced by spaces/newlines.
Tokenizer	Preprocessed text	Ordered `Token` records containing type, lexeme, line, column.
Brace detector	`vector<Token>`	Half-open `{...}` index intervals.
Function detector	Tokens + brace intervals	Restricted subset of function-body intervals.
Type-1 comparator	Two token vectors + intervals	Boolean exact body-level match.
Match collector	Two groups of function intervals	`Type1BodyMatch` records.
Reporter	Token vectors + matches + filenames	Human-readable line-based candidate evidence.

Part	Input	Output / observed behavior
Application	File or directory path	Token counts, region counts; folder pairwise candidate results.

What Phase 2 does NOT claim

It does not parse arbitrary C++ syntax, normalize renamed identifiers, identify Type-2/Type-3/Type-4 clones, train machine-learning models, compute a stylometric profile, or decide plagiarism. The approved future project scope is broader than the verified Phase 2 build.

2. Pipeline architecture and C++ reading kit

2.1 Execution order

`main()` -> `runCloneScope()`

`std::ifstream` -> `std::ostringstream` -> source string

`removeComments` -> `tokenize`

`findBraceRegions` -> `findFunctionBodyRegions`

`AnalyzedSourceFile` records stored in `std::vector`

For each file pair `i < j`: `findType1BodyMatches`

If nonempty: `printType1BodyMatches` -> `std::cout`

Each arrow is a call or data transfer already present in the supplied application. The reports are printed directly; they are not saved as CSV or JSON by this phase.

2.2 Syntax dictionary for this project

C++ construct	Literal example from code	Meaning
Struct	`struct CodeRegion { ... };`	A user-defined group of related fields.
Vector	`std::vector<Token>`	Growable sequence of token objects.
Const reference	`const std::vector<Token> &tokens`	Read-only view of an existing vector without a full copy.
Size type	`std::size_t`	Unsigned type used for sizes and indexes.
Range loop	`for (const CodeRegion &region : braceRegions)`	Visit each region by read-only reference.
Index loop	`for (std::size_t i=0; i<...; ++i)`	Visit elements by numeric index.
Aggregate init	`CodeRegion{start, i + 1}`	Initialize fields in declaration order.
Member access	`region.beginTokenIndex`	Read a field from an object.
Logical OR	`a || b`	True if either condition is true; uses short circuit.
Logical AND	`a && b`	True only if both conditions are true; uses short circuit.
Not	`!matches.empty()`	Invert bool; here, at least one match.
Early exit	`continue` / `return false`	Skip a loop iteration / stop a function.
Scope alias	`namespace fs = std::filesystem;`	Use `fs::path` as shorter spelling.
Stream reference	`std::ostream &output`	Caller selects console or capture stream.

2.3 Token and region data models

Actual source: src/tokenizer.hpp · lines 1-27 (as supplied)

```
 1 | #pragma once
 2 |
 3 | #include <cstddef>
 4 | #include <string>
 5 | #include <vector>
 6 |
 7 | enum class TokenType
 8 | {
 9 |     Keyword,
10 |     Identifier,
11 |     Number,
12 |     StringLiteral,
13 |     CharLiteral,
14 |     Operator,
15 |     Punctuation,
16 |     Unknown
17 | };
18 |
19 | struct Token
20 | {
21 |     TokenType type;
22 |     std::string lexeme;
```

```
23 |     std::size_t line;
24 |     std::size_t column;
25 | };
26 |
27 | std::vector<Token> tokenize(const std::string &source);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: src/code_region.hpp · lines 1-15 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <cstddef>
 5 | #include <vector>
 6 | #include "tokenizer.hpp"
 7 |
 8 | struct CodeRegion
 9 | {
10 |     std::size_t beginTokenIndex;
11 |     std::size_t endTokenIndex;
12 | };
13 |
14 | std::vector<CodeRegion> findBraceRegions(
15 |     const std::vector<Token> &tokens);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

A `Token` has what (`type`), exact characters (`lexeme`), and where (`line`, `column`). A `CodeRegion` has where in the token vector (`beginTokenIndex`, `endTokenIndex`), not source-line fields. The reporter converts index ranges back to source lines using tokens.

> **THE SINGLE MOST IMPORTANT RANGE RULE**
>
> All Phase 2 regions use the half-open interval `[beginTokenIndex, endTokenIndex)`: the opening `{` is included, but the numeric `endTokenIndex` points just after the closing `}`. So length is `end - begin`, and the last token is `tokens[end - 1]`.

3. Preprocessing and tokenization recap

3.1 Why comment removal comes first

If comments remain, words inside `//` or `/* ... */` would be tokenized as if they were program code. The supplied `removeComments` uses five `ScanState` values: `Normal`, `LineComment`, `BlockComment`, `StringLiteral`, `CharLiteral`. It replaces supported comment characters with spaces rather than deleting lines, while preserving newline boundaries.

Actual source: src/preprocessor.cpp · lines 5-20 (as supplied)

```
 5 | enum class ScanState
 6 | {
 7 |     Normal,
 8 |     LineComment,
 9 |     BlockComment,
10 |     StringLiteral,
11 |     CharLiteral
12 | };
13 |
14 | // Basic comment removal using a lexical state machine.
15 | // Preserves source positions for supported comment cases.
16 |
17 | std::string removeComments(const std::string &source)
18 | {
19 |     ScanState state = ScanState::Normal;
20 |     std::string result;
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: src/preprocessor.cpp · lines 24-44 (as supplied)

```
24 |     for (std::size_t i = 0; i < source.size(); ++i)
25 |     {
26 |         char current = source[i];
27 |
28 |         if (state == ScanState::Normal)
29 |         {
30 |             if (current == '/' && i + 1 < source.size() &&
31 |                 source[i + 1] == '/')
32 |             {
33 |                 state = ScanState::LineComment;
34 |                 result += "  ";
35 |                 ++i;
36 |             }
37 |
38 |             else if (current == '/' && i + 1 < source.size() &&
39 |                      source[i + 1] == '*')
40 |             {
41 |                 state = ScanState::BlockComment;
42 |                 result += "  ";
43 |                 ++i;
44 |             }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: src/preprocessor.cpp · lines 61-91 (as supplied)

```
61 |         else if (state == ScanState::LineComment)
62 |         {
63 |             if (current == '\n')
64 |             {
65 |                 result += '\n';
66 |                 state = ScanState::Normal;
67 |             }
68 |             else
69 |             {
70 |                 result += ' ';
71 |             }
72 |         }
73 |
74 |         else if (state == ScanState::BlockComment)
75 |         {
76 |             if (current == '*' && i + 1 < source.size() &&
77 |                 source[i + 1] == '/')
78 |             {
79 |                 result += "  ";
80 |                 state = ScanState::Normal;
81 |                 ++i;
82 |             }
```

```
83 |             else if (current == '\n')
84 |             {
85 |                 result += '\n';
86 |             }
87 |             else
88 |             {
89 |                 result += ' ';
90 |             }
91 |         }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 5–12: `enum class` defines a strongly typed list of states. Qualify names as `ScanState::Normal`; a bare `Normal` is not automatically in scope.

Lines 17–20: The function takes `const std::string &`: caller text cannot be modified through this reference. `result` begins empty, and the scanner starts in `Normal`.

Lines 24–26: The `for` loop walks the original characters. `current` holds one source character at a time.

Lines 30–35: Two `/` characters begin a line comment. Two spaces preserve character positions for those two characters; `++i` avoids reprocessing the second slash.

Lines 38–44: `/*` begins a block comment; the same two-character skip is used.

Lines 61–71: Within `LineComment`, newline is preserved and resets `Normal`; non-newline comment characters are replaced by spaces.

Lines 74–91: Within `BlockComment`, `*/` closes the comment. Other characters become spaces except
, which remains
.

Lines 93–149: Quoted literal states preserve ordinary literal content and basic escapes. An unclosed ordinary literal reaching newline is recovered at that newline.

3.2 What the tokenizer preserves

Actual source: src/tokenizer.hpp · lines 1-27 (as supplied)

```
 1 | #pragma once
 2 |
 3 | #include <cstddef>
 4 | #include <string>
 5 | #include <vector>
 6 |
 7 | enum class TokenType
 8 | {
 9 |     Keyword,
10 |     Identifier,
11 |     Number,
12 |     StringLiteral,
13 |     CharLiteral,
14 |     Operator,
15 |     Punctuation,
16 |     Unknown
17 | };
18 |
19 | struct Token
20 | {
21 |     TokenType type;
22 |     std::string lexeme;
```

```
23 |     std::size_t line;
24 |     std::size_t column;
25 | };
26 |
27 | std::vector<Token> tokenize(const std::string &source);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

`enum class TokenType` separates categories such as `Identifier`, `Keyword`, `Number`, `Operator`, and `Punctuation`. The tokenizer stores the source substring as `lexeme`; this exact text is crucial, because `+` and `-` are both operators but cannot be Type-1 equivalent.

Actual source: src/tokenizer.cpp · lines 13-49 (as supplied)

```
13 | std::vector<Token> tokenize(const std::string &source)
14 | {
15 |     std::vector<Token> tokens;
16 |
17 |     // Common C/C++ keywords; the list will be expanded later.
18 |     static const std::unordered_set<std::string> keywords = {
19 |         "int", "float", "double", "char", "void",
20 |         "if", "else", "for", "while", "return",
21 |         "bool", "const", "class", "struct"};
22 |
23 |     // Single-character structural punctuation supported by the tokenizer.
24 |     static const std::string punctuation = "(){}[],;";
25 |
26 |     // Common single-character operators.
27 |     // Used when no supported two-character operator matches.
28 |
29 |     static const std::string singleCharOperators = "+-*/%&=!<>|^~.";
30 |
31 |     // Supported two-character operators.
32 |     // Checked before single-character operators to preserve longest matches.
33 |
34 |     static const std::unordered_set<std::string> twoCharOperators = {
```

```
35 |         "==", "++", "--",
36 |         "+=", "-=", "*=", "/=",
37 |         "%=", "&=", "|=", "^=",
38 |         "!=", "<=", ">=", "&&", "||",
39 |         "::", "->"};
40 |
41 |     // Track the current character and source location.
42 |
43 |     std::size_t index = 0;
44 |     std::size_t line = 1;
45 |     std::size_t column = 1;
46 |
47 |     while (index < source.size())
48 |     {
49 |         char current = source[index];
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: src/tokenizer.cpp · lines 258-335 (as supplied)

```
258 |         // Recognize single-character structural punctuation.
259 |         if (punctuation.find(current) != std::string::npos)
260 |         {
261 |             // Store the punctuation and its source location.
262 |             tokens.push_back(Token{
263 |                 TokenType::Punctuation,
264 |                 source.substr(index, 1),
265 |                 line,
266 |                 column});
267 |
268 |             ++index;
269 |             ++column;
270 |
271 |             continue;
272 |         }
273 |
274 |         // Recognize supported two-character operators first.
275 |         if (index + 1 < source.size())
276 |         {
277 |             std::string pair = source.substr(index, 2);
278 |
279 |             if (twoCharOperators.find(pair) != twoCharOperators.end())
```

```
280 |             {
281 |                 // Preserve the complete operator and its starting location.
282 |                 tokens.push_back(Token{
283 |                     TokenType::Operator,
284 |                     pair,
285 |                     line,
286 |                     column});
287 |
288 |                 index += 2;
289 |                 column += 2;
290 |
291 |                 continue;
292 |             }
293 |         }
294 |
295 |         // Recognize a basic single-character operator.
296 |         if (singleCharOperators.find(current) != std::string::npos)
297 |         {
298 |             // Preserve the operator and its source location.
299 |             tokens.push_back(Token{
300 |                 TokenType::Operator,
301 |                 source.substr(index, 1),
```

```
302 |                 line,
303 |                 column});
304 |
305 |             ++index;
306 |             ++column;
307 |
308 |             continue;
309 |         }
310 |
311 |         // Preserve unsupported non-whitespace characters.
312 |         if (!std::isspace(static_cast<unsigned char>(current)))
313 |         {
314 |             tokens.push_back(Token{
315 |                 TokenType::Unknown,
316 |                 source.substr(index, 1),
317 |                 line,
318 |                 column});
319 |         }
320 |
321 |         if (current == '\n')
322 |         {
323 |             ++line;
```

```
324 |             column = 1;
325 |         }
326 |         else
327 |         {
328 |             ++column;
329 |         }
330 |
331 |         ++index;
332 |     }
333 |
334 |     return tokens;
335 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 13–15: `tokenize` receives preprocessed text and creates an empty `std::vector<Token>` result.

Lines 18–21: A fixed `unordered_set` lists selected keywords, not the complete C++ keyword set. This limitation influences function recognition.

Lines 24–39: Punctuation, one-character operators, and selected two-character operators are explicit lookup lists.

Lines 43–49: `index` locates the current character, and `line` / `column` begin at one, so printed locations are human-readable.

Lines 258–272: Matching punctuation becomes a `Punctuation` token; index and column are incremented once.

Lines 274–293: Two-character operators are checked before one-character operators (`==` must not become two separate `=` tokens).

Lines 295–309: Selected single-character operators are recognized next.

Lines 311–319: Unsupported non-whitespace characters become `Unknown` tokens, not silently discarded.

Lines 321–334: A newline advances line and resets column to one; otherwise column increments. At the end return `tokens`.

3.3 Concrete source-position trace

Three-line source example

```
int a; /* note */ int b;
// comment
int c;
```

After comment removal, `int b;` still starts at physical line 1, column 19, and `int c;` still starts at line 3, column 1. These assertions appear in the supplied `pipeline_smoke.cpp` (which additionally needs `samples/hello.c`).

3.4 Lexeme versus token type

Lexeme	Type	Type-1 implication
`+`	Operator	Different from `-` despite same type.
`-`	Operator	Exact lexeme mismatch rejects.
`10`	Number	Different from `20`.
`a`	Identifier	Different from `x`.
`return`	Keyword	Identical keyword matches.
`{` / `}`	Punctuation	Brace tokens are part of compared body intervals.

4. CodeRegion: brace matching as a stack

4.1 Actual implementation

Actual source: src/code_region.cpp · lines 1-27 (as supplied)

```
 1 |
 2 | #include "code_region.hpp"
 3 |
 4 | std::vector<CodeRegion> findBraceRegions(
 5 |     const std::vector<Token> &tokens)
 6 | {
 7 |     std::vector<CodeRegion> regions;
 8 |     std::vector<std::size_t> openingBraceIndices;
 9 |
10 |     for (std::size_t i = 0; i < tokens.size(); ++i)
11 |     {
12 |         if (tokens[i].lexeme == "{")
13 |         {
14 |             openingBraceIndices.push_back(i);
15 |         }
16 |         else if (tokens[i].lexeme == "}" &&
17 |                  !openingBraceIndices.empty())
18 |         {
19 |             std::size_t start = openingBraceIndices.back();
20 |             openingBraceIndices.pop_back();
21 |
22 |             regions.push_back(CodeRegion{start, i + 1});
```

```
23 |         }
24 |     }
25 |
26 |     return regions;
27 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 2: Including `code_region.hpp` supplies the `CodeRegion`, `Token`, and vector declarations.

Lines 4–6: `findBraceRegions` takes a read-only token vector and returns a vector of intervals.

Lines 7: `regions` accumulates completed matching brace pairs.

Lines 8: `openingBraceIndices` acts as a stack implemented with a vector of token indexes.

Lines 10: Walk token positions from zero to `tokens.size()-1`; `++i` advances after every iteration.

Lines 12–15: When the lexeme is `{`, push its token index onto the stack. The last opened brace is the first to close.

Lines 16–18: Only a closing `}` with a nonempty stack can produce a matched region; an unmatched `}` is ignored.

Lines 19: `.back()` reads the most recently pushed opening-brace position.

Lines 20: `.pop_back()` removes that opening position because the brace pair is now complete.

Lines 22: `CodeRegion{start, i + 1}` stores half-open bounds. Using `i + 1` includes the closing brace at `i`.

Lines 26: Return completed matched intervals. Unmatched opening braces remain on the stack and produce no region.

4.2 Worked nested-brace trace (18 tokens)

```
int main() { if (1) { return 1; } return 0; }
```

Index	Token	Stack after token	Region emitted
0–3	`int main ( )`	`[]`	none
4	`{`	`[4]`	none
5–8	`if ( 1 )`	`[4]`	none
9	`{`	`[4, 9]`	none
10–12	`return 1 ;`	`[4, 9]`	none
13	`}`	`[4]`	`[9, 14)`
14–16	`return 0 ;`	`[4]`	none
17	`}`	`[]`	`[4, 18)`

The inner region is appended before the outer region because the inner closing brace appears first. The test explicitly asserts this order. The code does not attempt to form an AST or classify all brace blocks.

4.3 Edge cases in the supplied test

`findBraceRegions` is checked against: ordinary function body, nested braces, no braces, unmatched opening `{`, unmatched closing `}`, and empty `{}`. For the empty block, the interval is `[0, 2)`, not an empty token interval.

> **COMMON EXAM QUESTION**
>
> Why use a stack? Nested braces obey last opened, first closed order (LIFO). A queue would match an outer opening brace to an inner closing brace incorrectly.

5. Restricted function-body recognition

5.1 Exact source listing

Actual source: src/function_region.cpp · lines 1-54 (as supplied)

```
 1 |
 2 | #include "function_region.hpp"
 3 |
 4 | std::vector<CodeRegion> findFunctionBodyRegions(
 5 |     const std::vector<Token> &tokens)
 6 | {
 7 |     std::vector<CodeRegion> functionRegions;
 8 |     std::vector<CodeRegion> braceRegions = findBraceRegions(tokens);
 9 |
10 |     for (const CodeRegion &region : braceRegions)
11 |     {
12 |         std::size_t braceIndex = region.beginTokenIndex;
13 |
14 |         if (braceIndex == 0 || tokens[braceIndex - 1].lexeme != ")")
15 |         {
16 |             continue;
17 |         }
18 |
19 |         std::size_t depth = 0;
20 |         std::size_t openParenIndex = tokens.size();
21 |
22 |         for (std::size_t i = braceIndex; i > 0; --i)
```

```
23 |         {
24 |             std::size_t index = i - 1;
25 |
26 |             if (tokens[index].lexeme == ")")
27 |             {
28 |                 ++depth;
29 |             }
30 |             else if (tokens[index].lexeme == "(")
31 |             {
32 |                 --depth;
33 |
34 |                 if (depth == 0)
35 |                 {
36 |                     openParenIndex = index;
37 |                     break;
38 |                 }
39 |             }
40 |         }
41 |
42 |         if (openParenIndex == tokens.size() || openParenIndex == 0)
43 |         {
44 |             continue;
```

```
45 |         }
46 |
47 |         if (tokens[openParenIndex - 1].type == TokenType::Identifier)
48 |         {
49 |             functionRegions.push_back(region);
50 |         }
51 |     }
52 |
53 |     return functionRegions;
54 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

5.2 Line-by-line interpretation

Lines 4–8: Create the result vector and call the brace matcher, so function recognition is a filter over all recognized brace regions.

Lines 10–12: Visit each brace region by `const` reference. Its `beginTokenIndex` identifies the opening `{`.

Lines 14–17: Guard against `braceIndex==0` before accessing the previous token; require the token just before `{` to be `)`. A `continue` skips nonmatching braces.

Lines 19: Initialize nesting `depth` to zero for a backward scan of parentheses.

Lines 20: Set `openParenIndex = tokens.size()` as a sentinel meaning "not found yet".

Lines 22–24: Walk backward with `i > 0`, computing `index=i-1`. This avoids writing `--i` on an unsigned index that could underflow after reaching zero.

Lines 26–29: Every encountered `)` increases nesting depth while scanning backward.

Lines 30–39: Every encountered `(` decreases depth. When depth reaches zero, this `(` matches the immediately preceding `)`; save its index and `break`.

Lines 42–45: Reject when no matching `(` was found or when the opening parenthesis is at index zero, leaving no preceding name token.

Lines 47–50: Classify as a supported function body only if the token immediately before `(` has `TokenType::Identifier`. Append the existing region.

Lines 53: Return the selected regions. This remains a heuristic rather than a syntax tree.

5.3 Trace: a function containing `if`

```
int calculate(int x) { if (x > 0) { return x; } return 0; }
```

Brace	Token before `{`	Token before matching `(`	Decision
Outer body	`)`	`calculate` (Identifier)	Accept as supported function body.
Inner `if` block	`)`	`if` (Keyword)	Reject as a function body.

The supplied test expects two brace regions but one function region, with outer function interval starting at token index 6.

5.4 Cases verified and cases not guaranteed

Case	Supplied-test observation
Two normal functions	Two function regions recognized.
Nested `while` block	`while` block not added as a separate function.
Lambda `[](int x) { ... }`	Brace exists but function-regions result empty in supplied test.
Namespace with inner function	Only the inner function selected.
Braced initializer in function	Initializer brace excluded; function body retained.
Advanced C++ signatures	No complete syntax coverage asserted.

IMPORTANT HEURISTIC LIMITATION

The tokenizer has a short keyword set. For example `switch` and `catch` are not in that set in this snapshot, so a `switch (...) { ... }` or `catch (...) { ... }` can be mistaken for a function body by the identifier-before-parenthesis rule. Also, trailing-return-type functions such as `auto f() -> int { ... }` do not satisfy the immediate-`)` guard. Do not claim this is a full C++ function parser.

6. Exact Type-1 function-body matching

6.1 Data contract: two regions for one match

Actual source: src/type1_clone.hpp · lines 1-23 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <vector>
 5 | #include "code_region.hpp"
 6 |
 7 | struct Type1BodyMatch
 8 | {
 9 |     CodeRegion firstRegion;
10 |     CodeRegion secondRegion;
11 | };
12 |
13 | bool areType1BodyCandidates(
14 |     const std::vector<Token> &firstTokens,
15 |     const CodeRegion &firstRegion,
16 |     const std::vector<Token> &secondTokens,
17 |     const CodeRegion &secondRegion);
18 |
19 | std::vector<Type1BodyMatch> findType1BodyMatches(
20 |     const std::vector<Token> &firstTokens,
21 |     const std::vector<CodeRegion> &firstRegions,
22 |     const std::vector<Token> &secondTokens,
```

```
23 |     const std::vector<CodeRegion> &secondRegions);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

A `Type1BodyMatch` contains `firstRegion` and `secondRegion`, preserving where the two matching bodies occurred in their respective token vectors. It does not contain plagiarism confidence, student identity, or source filenames.

6.2 Actual implementation

Actual source: src/type1_clone.cpp · lines 1-70 (as supplied)

```
 1 |
 2 | #include "type1_clone.hpp"
 3 |
 4 | bool areType1BodyCandidates(
 5 |     const std::vector<Token> &firstTokens,
 6 |     const CodeRegion &firstRegion,
 7 |     const std::vector<Token> &secondTokens,
 8 |     const CodeRegion &secondRegion)
 9 | {
10 |     if (firstRegion.beginTokenIndex >= firstRegion.endTokenIndex ||
11 |         secondRegion.beginTokenIndex >= secondRegion.endTokenIndex ||
12 |         firstRegion.endTokenIndex > firstTokens.size() ||
13 |         secondRegion.endTokenIndex > secondTokens.size())
14 |     {
15 |         return false;
16 |     }
17 |
18 |     std::size_t firstLength =
19 |         firstRegion.endTokenIndex - firstRegion.beginTokenIndex;
20 |
21 |     std::size_t secondLength =
22 |         secondRegion.endTokenIndex - secondRegion.beginTokenIndex;
```

```
23 |
24 |     if (firstLength != secondLength)
25 |     {
26 |         return false;
27 |     }
28 |
29 |     for (std::size_t i = 0; i < firstLength; ++i)
30 |     {
31 |         const Token &first =
32 |             firstTokens[firstRegion.beginTokenIndex + i];
33 |
34 |         const Token &second =
35 |             secondTokens[secondRegion.beginTokenIndex + i];
36 |
37 |         if (first.type != second.type ||
38 |             first.lexeme != second.lexeme)
39 |         {
40 |             return false;
41 |         }
42 |     }
43 |
44 |     return true;
```

```
 45 | }
 46 |
 47 | std::vector<Type1BodyMatch> findType1BodyMatches(
 48 |     const std::vector<Token> &firstTokens,
 49 |     const std::vector<CodeRegion> &firstRegions,
 50 |     const std::vector<Token> &secondTokens,
 51 |     const std::vector<CodeRegion> &secondRegions)
 52 | {
 53 |     std::vector<Type1BodyMatch> matches;
 54 |
 55 |     for (const CodeRegion &firstRegion : firstRegions)
 56 |     {
 57 |         for (const CodeRegion &secondRegion : secondRegions)
 58 |         {
 59 |             if (areType1BodyCandidates(
 60 |                     firstTokens, firstRegion,
 61 |                     secondTokens, secondRegion))
 62 |             {
 63 |                 matches.push_back(
 64 |                     Type1BodyMatch{firstRegion, secondRegion});
 65 |             }
 66 |         }
```

```
 67 |     }
 68 |
 69 |     return matches;
 70 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

6.3 Detailed line-by-line explanation

Lines 4–9: Function `areType1BodyCandidates` takes two token vectors and one proposed body interval for each; all inputs are read-only references.

Lines 10–16: Reject empty/inverted ranges and ranges whose end exceeds the corresponding token-vector size. This avoids out-of-bounds access.

Lines 18–22: Calculate token lengths using the half-open interval formula `end - begin`.

Lines 24–27: If token counts differ, exact Type-1 body equality is impossible; reject without comparing lexemes.

Lines 29–36: Iterate corresponding offsets from zero to length minus one; derive references to the two current tokens.

Lines 37–42: Reject as soon as either token `type` or exact `lexeme` differs. `||` means one mismatch is sufficient.

Lines 44: Return true only after every token has passed both comparisons.

Lines 47–53: `findType1BodyMatches` returns a vector of match records, not a Boolean.

Lines 55–58: Nested range-based loops enumerate every possible function-body combination between these two source files.

Lines 59–66: Call the validator/comparator. If true, append a `Type1BodyMatch{firstRegion, secondRegion}` record.

Lines 69: Return the resulting list; an empty vector means no supported exact matching pair was found.

6.4 The 16-token `add()` example

```
int add(int a, int b) { return a + b; }
```

Token indexes	Values / explanation
0–8	`int, add, (, int, a, ,, int, b, )` - signature, not compared.
9	`{` - body begin index.
10–14	`return, a, +, b, ;` - body content.
15	`}` - last body token.
16	End-exclusive boundary; this is not a token in this 16-token vector.

The two simple fixtures both produce a body region `[9,16)`, which has 7 tokens: `{ return a + b; }`. Whitespace/comment variants preserve this token sequence.

6.5 Why these changes cause rejection

First	Second	Why no Type-1 candidate
`return a + b;`	`return a - b;`	`+` and `-` have different lexemes.
`return a + b;`	`return x + y;`	Identifier lexemes differ.
`return 10;`	`return 20;`	Number lexemes differ.
`return a+b;`	`return a + b; // comment`	Candidate can match after supported comment removal.

6.6 Match record and runtime output are not verdicts

The code checks equality of token types and lexemes, not authorship. Even short assignment-template function bodies can match. A match record should be read as evidence for review, not an allegation.

7. The explainable reporting module

7.1 Interface and implementation

Actual source: src/type1_report.hpp · lines 1-16 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <ostream>
 5 | #include <string>
 6 | #include <vector>
 7 |
 8 | #include "type1_clone.hpp"
 9 |
10 | void printType1BodyMatches(
11 |     std::ostream &output,
12 |     const std::string &firstFileName,
13 |     const std::vector<Token> &firstTokens,
14 |     const std::string &secondFileName,
15 |     const std::vector<Token> &secondTokens,
16 |     const std::vector<Type1BodyMatch> &matches);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: src/type1_report.cpp · lines 1-57 (as supplied)

```
 1 |
 2 | #include "type1_report.hpp"
 3 |
 4 | void printType1BodyMatches(
 5 |     std::ostream &output,
 6 |     const std::string &firstFileName,
 7 |     const std::vector<Token> &firstTokens,
 8 |     const std::string &secondFileName,
 9 |     const std::vector<Token> &secondTokens,
10 |     const std::vector<Type1BodyMatch> &matches)
11 | {
12 |     output << "CloneScope Type-1 body-level candidate report\n";
13 |
14 |     std::size_t reportedCount = 0;
15 |
16 |     for (const Type1BodyMatch &match : matches)
17 |     {
18 |         const CodeRegion &first = match.firstRegion;
19 |         const CodeRegion &second = match.secondRegion;
20 |
21 |         if (first.beginTokenIndex >= first.endTokenIndex ||
22 |             first.endTokenIndex > firstTokens.size() ||
```

```
23 |             second.beginTokenIndex >= second.endTokenIndex ||
24 |             second.endTokenIndex > secondTokens.size())
25 |         {
26 |             output << "Warning: skipped invalid match boundaries.\n";
27 |             continue;
28 |         }
29 |
30 |         const Token &firstStart =
31 |             firstTokens[first.beginTokenIndex];
32 |
33 |         const Token &firstEnd =
34 |             firstTokens[first.endTokenIndex - 1];
35 |
36 |         const Token &secondStart =
37 |             secondTokens[second.beginTokenIndex];
38 |
39 |         const Token &secondEnd =
40 |             secondTokens[second.endTokenIndex - 1];
41 |
42 |         ++reportedCount;
43 |
44 |         output << "\nCandidate #" << reportedCount << "\n";
```

```
45 |
46 |         output << firstFileName << ": lines "
47 |                << firstStart.line << " to " << firstEnd.line << "\n";
48 |
49 |         output << secondFileName << ": lines "
50 |                << secondStart.line << " to " << secondEnd.line << "\n";
51 |
52 |         output << "Evidence: identical body token types and lexemes.\n";
53 |         output << "Human review required; similarity is not proof of plagiarism.\n";
54 |     }
55 |
56 |     output << "\nReported candidates: " << reportedCount << "\n";
57 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

7.2 Line-by-line explanation

Lines 4–10: The function accepts a generic output stream, two names, two token vectors, and match records. `std::ostream &` lets a caller pass `std::cout` or `std::ostringstream`.

Lines 12: Write the report heading before the candidate loop.

Lines 14: Initialize `reportedCount`; this counts only match records passing the reporter boundary check.

Lines 16–19: Iterate matches and bind short read-only references to their two `CodeRegion` fields.

Lines 21–28: Skip empty/inverted/out-of-range intervals to avoid invalid token indexing. The reporter trusts the caller for actual equality; it does not re-run the Type-1 matcher.

Lines 30–40: Read the start token at `beginTokenIndex` and end token at `endTokenIndex-1` for each file.

Lines 42–44: Increase the accepted report count and print numbered candidate heading.

Lines 46–50: Write filename and the physical source lines stored in the starting and ending tokens.

Lines 52–53: Print the evidence description and essential warning about human review.

Lines 56: Print the number of actually reported records.

7.3 Verified output from the uploaded realistic fixture pair

```
CloneScope Type-1 body-level candidate report

Candidate #1
realistic_student_a.cpp: lines 7 to 9
realistic_student_b.cpp: lines 10 to 13
Evidence: identical body token types and lexemes.
Human review required; similarity is not proof of plagiarism.

Reported candidates: 1
```

The A and B line ranges differ because Student B has extra comment lines. Keeping original source positions lets the instructor open the correct locations even though comments were ignored for matching.

7.4 Testing a writer without printing

```
std::ostringstream reportBuffer;
printType1BodyMatches(reportBuffer, "realistic_student_a.cpp",
    realisticFirstTokens, "realistic_student_b.cpp",
    realisticSecondTokens, matches);
std::string reportText = reportBuffer.str();
assert(reportText.find("Reported candidates: 1") != std::string::npos);
```

`reportBuffer` is a memory-backed stream. `.str()` returns the text; `.find(...)` searches within it; `std::string::npos` means "not found". The `assert` succeeds only when the required text exists (provided assertions are enabled).

8. Application-level multi-file integration

8.1 Key application data model

Actual source: src/application.cpp · lines 26-35 (as supplied)

```
 26 | using namespace std;
 27 |
 28 | namespace fs = std::filesystem;
 29 |
 30 | struct AnalyzedSourceFile
 31 | {
 32 |     fs::path path;
 33 |     std::vector<Token> tokens;
 34 |     std::vector<CodeRegion> functionRegions;
 35 | };
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 26: `using namespace std;` makes unqualified `cout`, `ifstream` etc. legal within this translation unit; it is a convenience, not a requirement.

Lines 28: Alias `std::filesystem` as `fs`, so `fs::path` and `fs::directory_iterator` are shorter.

Lines 30–35: `AnalyzedSourceFile` groups one file path, copied tokens, and copied function-body intervals; this prevents mixing records from different files.

8.2 Input and directory discovery

Actual source: src/application.cpp · lines 37-79 (as supplied)

```
37 | int runCloneScope()
38 | {
39 |     cout << "================================\n";
40 |     cout << "        CloneScope v0.1\n";
41 |     cout << "================================\n";
42 |     cout << "Code Clone Analysis Tool\n";
43 |     cout << "Status: Development Started\n";
44 |
45 |     // Read a user-selected C/C++ source file
46 |     // Ask the user for a source file path
47 |     string filePath;
48 |
49 |     cout << "Enter source file path: ";
50 |     getline(cin, filePath);
51 |
52 |     fs::path sourcePath(filePath);
53 |
54 |     // Check whether the input path is a directory
55 |     if (fs::is_directory(sourcePath))
56 |     {
57 |         cout << "Folder detected successfully!\n";
58 |
```

```
59 |         std::vector<AnalyzedSourceFile> analyzedFiles;
60 |
61 |         // Visit each entry directly inside the folder
62 |         for (const auto &entry : fs::directory_iterator(sourcePath))
63 |         {
64 |             if (entry.is_regular_file() &&
65 |                 (entry.path().extension() == ".c" ||
66 |                  entry.path().extension() == ".cpp"))
67 |             {
68 |                 cout << entry.path() << '\n';
69 |                 // Open the discovered source file
70 |                 ifstream sourceFile(entry.path());
71 |
72 |                 // Handle a file-opening failure
73 |                 if (!sourceFile.is_open())
74 |                 {
75 |                     cerr << "Error: Could not open " << entry.path() << '\n';
76 |                     continue;
77 |                 }
78 |
79 |                 cout << "Source file opened successfully!\n";
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 37–43: `runCloneScope` prints a simple banner and returns an integer process exit code.

Lines 47–52: `getline(cin, filePath)` reads an entire path line, including spaces. `fs::path` stores it in filesystem-aware form.

Lines 55–59: Check for a directory; allocate `analyzedFiles` outside the per-file loop so records remain available after each iteration.

Lines 62–67: Visit direct child entries only; accept regular files whose extension is exactly `.c` or `.cpp`.

Lines 70–77: Attempt to open each discovered file. On failure, print an error and `continue` to the next directory entry.

8.3 Analyze once, store once

Actual source: src/application.cpp · lines 81-112 (as supplied)

```
 81 |                 // Read the complete discovered source file
 82 |                 std::ostringstream buffer;
 83 |                 buffer << sourceFile.rdbuf();
 84 |
 85 |                 std::string sourceCode = buffer.str();
 86 |
 87 |                 // Remove comments from this source file
 88 |                 std::string cleanedCode = removeComments(sourceCode);
 89 |
 90 |                 // Display the processed source code
 91 |                 cout << "Preprocessed source code:\n";
 92 |                 cout << cleanedCode << '\n';
 93 |
 94 |                 // Tokenize the current source file.
 95 |                 std::vector<Token> tokens = tokenize(cleanedCode);
 96 |
 97 |                 // Display the total token count for this file.
 98 |                 cout << "Total tokens: " << tokens.size() << '\n';
 99 |
100 |                 std::vector<CodeRegion> functionRegions =
101 |                     findFunctionBodyRegions(tokens);
102 |
```

```
103 |                 cout << "Supported function-body regions: "
104 |                      << functionRegions.size() << '\n';
105 |
106 |                 analyzedFiles.push_back(
107 |                     AnalyzedSourceFile{entry.path(), tokens, functionRegions});
108 |             }
109 |         }
110 |
111 |         cout << "\nAnalyzed source files stored: "
112 |              << analyzedFiles.size() << '\n';
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 82–85: `buffer << sourceFile.rdbuf()` transfers the entire file stream into a string stream; `.str()` obtains the source text.

Lines 88: Call the verified comment remover.

Lines 91–92: Print preprocessed source. This is verbose development output, not a formal report.

Lines 95–101: Tokenize once, count tokens, and identify supported function-body intervals.

Lines 106–107: Use aggregate initialization to copy the file path, token vector, and region vector into the permanent per-folder `analyzedFiles` vector.

Lines 111–112: After the directory loop, print the number of successfully stored records.

8.4 Enumerate unique file pairs and report

Actual source: src/application.cpp · lines 114-156 (as supplied)

```
114 |         std::size_t filePairCount = 0;
115 |
116 |         for (std::size_t i = 0; i < analyzedFiles.size(); ++i)
117 |         {
118 |             for (std::size_t j = i + 1; j < analyzedFiles.size(); ++j)
119 |             {
120 |                 const AnalyzedSourceFile &first = analyzedFiles[i];
121 |                 const AnalyzedSourceFile &second = analyzedFiles[j];
122 |
123 |                 cout << "Comparing files: "
124 |                      << first.path.filename()
125 |                      << " vs "
126 |                      << second.path.filename() << '\n';
127 |
128 |                 std::vector<Type1BodyMatch> matches =
129 |                     findType1BodyMatches(
130 |                         first.tokens,
131 |                         first.functionRegions,
132 |                         second.tokens,
133 |                         second.functionRegions);
134 |
135 |                 cout << "Type-1 body-level candidates: "
```

```
136 |                      << matches.size() << '\n';
137 |
138 |                 if (!matches.empty())
139 |                 {
140 |                     printType1BodyMatches(
141 |                         cout,
142 |                         first.path.filename().string(),
143 |                         first.tokens,
144 |                         second.path.filename().string(),
145 |                         second.tokens,
146 |                         matches);
147 |                 }
148 |
149 |                 ++filePairCount;
150 |             }
151 |         }
152 |
153 |         cout << "Total source-file pairs: "
154 |              << filePairCount << '\n';
155 |
156 |         return 0;
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Lines 114: Initialize the pair counter at zero.

Lines 116–119: The outer index `i` picks the first file. The inner index starts at `i+1` to avoid self-pairs and reversed duplicates.

Lines 120–121: Read file objects through `const` references so vectors are not copied for each comparison.

Lines 123–126: Print the two path basenames for concise terminal output.

Lines 128–133: Invoke the already-tested exact Type-1 body match collector with two token vectors and two region collections.

Lines 135–136: Print the number of matching region pairs for this single file pair.

Lines 138–147: If there is at least one match, call the reusable reporter. Converting `path.filename()` with `.string()` meets the `std::string` argument type.

Lines 149: One distinct pair has been considered; increment its count.

Lines 153–156: Print the final pair total, then return success from the directory path.

8.5 Single-file behavior is intentionally different

Actual source: src/application.cpp · lines 159-197 (as supplied)

```
159 |     // Open the selected source file
160 |     ifstream inputFile(sourcePath);
161 |
162 |     // Check whether the file opened successfully
163 |     if (!inputFile.is_open())
164 |     {
165 |         cerr << "Error: Could not open source file.\n";
166 |         return 1;
167 |     }
168 |
169 |     cout << "Source file opened successfully!\n";
170 |
171 |     // Read the complete source file into memory
172 |     std::ostringstream buffer;
173 |     buffer << inputFile.rdbuf();
174 |
175 |     std::string sourceCode = buffer.str();
176 |
177 |     // Remove comments from the source code
178 |     std::string cleanedCode = removeComments(sourceCode);
179 |
180 |     // Display the preprocessed source code.
```

```
181 |     cout << "Preprocessed source code:\n";
182 |     cout << cleanedCode;
183 |
184 |     // Convert the preprocessed source into tokens.
185 |     std::vector<Token> tokens = tokenize(cleanedCode);
186 |
187 |     // Display how many tokens were produced.
188 |     cout << "\nTotal tokens: " << tokens.size() << '\n';
189 |
190 |     std::vector<CodeRegion> functionRegions =
191 |         findFunctionBodyRegions(tokens);
192 |
193 |     cout << "Supported function-body regions: "
194 |          << functionRegions.size() << '\n';
195 |
196 |     return 0;
197 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

The single-file branch opens, reads, preprocesses, tokenizes, identifies function-body regions, and displays the counts. It returns without calling `findType1BodyMatches`, because one selected file alone does not define a pair of student submissions.

8.6 Pair-count proof

For `n` files, the first file is paired with `n-1` later files; the next with `n-2`, and so on, down to one. Sum = `(n-1)+(n-2)+...+1 = n(n-1)/2`. For the verified four-file positive fixture, this is `4*3/2 = 6`. For 40 files it would produce 780 pairs, but large-scale throughput has not been benchmarked.

> **VIVA DISTINCTION**
>
> Do not say 40 files have been stress-tested. The nested-loop pairing formula predicts 780 distinct file pairs; it says nothing by itself about end-to-end runtime on real student submissions.

9. Worked test walkthroughs

9.1 Positive fixture directory: 4 files

Dataset	Files	Distinct pairs	Reported body candidates
Positive	4	6	6
Negative	2	1	0
Mixed	3	3	1

These were the terminal results supplied during Phase 2 testing. The original positive fixtures all share one identical `add()` body; the realistic fixtures differ in their `main()` numeric literal, so the matching `add()` bodies are not proof that both programs are identical.

9.2 Negative fixtures: change a single operator

Actual source: tests/fixtures/type1/negative/negative_student_a.cpp · lines 1-5 (as supplied)

```
1 |
2 | int calculate(int a, int b)
3 | {
4 |     return a + b;
5 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Actual source: tests/fixtures/type1/negative/negative_student_b.cpp · lines 1-5 (as supplied)

```
1 |
2 | int calculate(int a, int b)
3 | {
4 |     return a - b;
5 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

They have the same function header and structure, but one body has `+` and the other `-`. Since lexemes differ, this pair produces zero restricted Type-1 body matches.

9.3 Mixed fixtures: one candidate, two rejected pairs

The mixed folder contains `positive_a.cpp`, `positive_b.cpp`, and `different.cpp`: both positive programs implement `add()` with an identical body; the third uses a subtraction body. Consequently the three unique comparisons yield `0`, `0`, and `1` (order may vary).

9.4 Exact line ranges in realistic fixture pair

Fixture	Opening brace	Return statement	Closing brace
realistic_student_a.cpp	line 7	line 8	line 9
realistic_student_b.cpp	line 10	line 12	line 13

This table is derived from the actual fixture files in the uploaded ZIP. Comment lines cause their brace locations to differ. The type and lexeme token comparisons still succeed for the `add()` body.

9.5 The seven regression smoke programs

Test source	Focus
`preprocessor_smoke.cpp`	Comment removal and preservation of ordinary literals.
`tokenizer_smoke.cpp`	Supported token categories, locations, recovery.
`pipeline_smoke.cpp`	Actual sample-file read; preprocessing + tokenization.
`code_region_smoke.cpp`	Half-open ranges, nesting, missing/unmatched braces.
`function_region_smoke.cpp`	Function bodies versus control blocks, lambdas, namespace and initializer.
`type1_clone_smoke.cpp`	Matching, operator/identifier/literal differences, invalid/empty ranges.
`type1_file_pipeline_smoke.cpp`	Read fixtures, match, report file paths and exact line ranges.

9.6 Practical build and run commands (PowerShell)

Build and launch the main application

```
g++ -std=c++17 -Wall -Wextra src/main.cpp src/application.cpp src/preprocessor.cpp src/token
  ... izer.cpp src/code_region.cpp src/function_region.cpp src/type1_clone.cpp src/type1_rep
  ... ort.cpp -o clonescope.exe
.\clonescope.exe
```

When asked for a path, enter `tests/fixtures/type1`, `tests/fixtures/type1/negative`, or `tests/fixtures/type1/mixed`. Run from the repository root so relative fixture paths resolve.

Build one integrated smoke test

```
g++ -std=c++17 -Wall -Wextra tests/type1_file_pipeline_smoke.cpp src/preprocessor.cpp src/to
  ... kenizer.cpp src/code_region.cpp src/function_region.cpp src/type1_clone.cpp src/type1_
  ... report.cpp -o type1_file_pipeline_smoke.exe
.\type1_file_pipeline_smoke.exe
Write-Host "Exit code: $LASTEXITCODE"
```

Each smoke test has its own `main()`; therefore test executables are built separately rather than combining test `.cpp` files into one binary.

> **BUNDLE REPRODUCIBILITY NOTE**
>
> The ZIP you uploaded includes `tests/pipeline_smoke.cpp` but not the referenced `samples/hello.c`. The pipeline smoke test hardcodes `samples/hello.c` and asserts 28 tokens. It passed on your original project earlier; when running from this ZIP alone, restore the original `samples/hello.c` fixture before expecting that test to run successfully. The other six test programs do not have this missing file dependency.

9.7 Assertions: the exact guarantee

`assert(condition)` terminates the program when a checked condition is false in a normal debug build. A printed success message means control reached that line; an exit code of zero indicates normal termination. Assertions can be disabled by defining `NDEBUG`, so do not compile these tests with `-DNDEBUG` if you want their checks active.

10. Complexity, accuracy boundaries, and risks

10.1 Cost model

Operation	Typical / worst-case discussion
Preprocessing	One pass over `C` source characters: O(C) time and O(C) output.
Tokenization	Primarily a linear scan over source characters; lookup and substring work adds implementation-level costs, usually near O(C).
Brace regions	Single walk over `T` tokens: O(T) time, O(T) worst-case stack and region storage.
Function regions	Scans brace regions and may scan backward through parentheses per region. Worst case O(R*T) for `R` brace regions, beyond cost of brace detection.
One body pair	O(K) token comparisons for K tokens, with lexeme-string comparisons also proportional to the characters actually compared.
One file pair	If files have F and G recognized bodies, up to F*G body comparisons.
N files	N(N-1)/2 file pairs; matching cost depends on function counts and token-body lengths.
In-memory records	Retains token vectors and region vectors for all successfully scanned files; current push-back copies vectors.

10.2 Why candidate counts are not plagiarism accuracy

A candidate count is a count of matching body-region pairs, not identical files, student pairs with proven copying, or calibrated percentage scores. A pair of files might yield multiple matches if multiple bodies share an exact token sequence. Even a trivial standard function can match across many independent submissions.

10.3 Concrete implementation boundaries

• Function recognition is a lightweight token heuristic, not a C++ grammar parser. Templates, qualifiers, trailing return types, macros and unusual function syntax are not systematically supported.

• The tokenizer recognizes only a small explicit keyword list. Words like `switch` and `catch` can be labeled `Identifier` in this snapshot, potentially leading the function-region heuristic to treat their brace blocks as functions.

• `findBraceRegions` skips unmatched closing braces and does not emit unmatched opening braces. It does not provide compiler diagnostics.

• Type-1 body matching includes `{` and `}` but excludes function names, signatures and return types. Consequently distinct functions can match if bodies match.

• Whitespace and supported comments are ignored; renamed identifiers, changed literals and changed operators are generally *not* ignored.

• Comment removal and tokenization do not implement all C++ lexical rules: raw string literals, full preprocessor directives, macro expansion, and line splicing remain unsupported.

• Directory traversal is non-recursive; only exactly lowercase `.c`/`.cpp` extensions are accepted. File ordering is not guaranteed by directory iterator.

• The application uses basename strings for report display; different subpaths with the same basename would require better disambiguation if scanning becomes recursive.

• Filesystem operations can fail; the current code handles failed file opening but does not comprehensively catch filesystem exceptions or report partial file reads.

• The reporter checks range shape but does not re-validate token equality if a caller fabricates a `Type1BodyMatch` record.

• The test set covers selected fixtures only. It cannot establish general accuracy, false-positive rate, or classroom-scale speed.

• Stylometry, Types 2–4, CSV/JSON export and automated academic misconduct decisions are not part of the verified Phase 2 implementation.

> **INTERPRETATION RULE**
>
> The defensible statement is: "CloneScope reports restricted Type-1 function-body candidate matches with explainable source locations, which an instructor must review." The indefensible statement is: "CloneScope proves plagiarism."

11. Git workflow and supervisor viva

11.1 What was completed in the Phase 2 branch

The project history discussed during development used feature branch `feat/function-block-representation`. Five meaningful local commits were reported, covering brace recognition, function detection, initial Type-1 comparison, reusable reporting, and application integration with test fixtures. The later documentation update and PDF manual should be committed before publication.

Checkpoint	Git short SHA reported during development
Brace-region detection	`f47d206`
Function-body detection	`dde94fd`
Type-1 comparator	`79f8811`
Explainable reporting	`64744ed`
Folder integration and test fixtures	`a8973b1`

11.2 Safe final documentation and PR commands

Illustrative sequence - run after copying the PDF into docs/learning-notes/

```
git status -sb
git diff -- README.md
git add README.md docs/phase2-function-block.md docs/learning-notes/CloneScope_Phase2_Functi
  ... on_Block_Type1_Learning_Manual.pdf
git diff --cached --check
git diff --cached --stat
git commit -m "docs: add Phase 2 learning manual and refresh README"
git status -sb
git push -u origin feat/function-block-representation
```

If `docs/phase2-function-block.md` was already committed, re-adding it is harmless; Git stages no differences. Review the actual status and staged diff before committing. Publishing does not itself merge into `main`; open a PR from `feat/function-block-representation` into `main`, review files and checks, merge, then verify local/remote branch state.

11.3 Viva practice: high-value questions and answers

Q1. What exactly does Phase 2 do?

Answer: It identifies restricted function-body regions from C/C++ tokens, compares them for exact Type-1 body-token equality across file pairs, and reports source locations for human review.

Q2. Why tokenize before clone matching?

Answer: Tokenization represents relevant code as typed lexemes, letting supported whitespace/comment differences be ignored.

Q3. What is a lexeme?

Answer: The exact text of a token, such as `+`, `add`, or `10`.

Q4. Why store token types AND lexemes?

Answer: The operator tokens `+` and `-` share a category but have different exact spellings; Type-1 must distinguish them.

Q5. What is a CodeRegion?

Answer: A pair of token indices `[begin,end)` delimiting a recognized brace block.

Q6. Why is `endTokenIndex` exclusive?

Answer: Length becomes `end-begin`, slicing is consistent, and the final included token is at `end-1`.

Q7. Why a stack for braces?

Answer: Nested blocks follow last-opened-first-closed pairing.

Q8. What happens to unmatched braces?

Answer: A closing brace without an opener is ignored; an opener without a closing brace does not emit a complete region.

Q9. How is a function body recognized?

Answer: The heuristic looks for a brace preceded by `)`, scans backward for matching `(`, and requires an Identifier just before that parenthesis.

Q10. Can it misidentify a switch block?

Answer: Yes. In this tokenizer snapshot, `switch` is not in the small keyword set and could be labeled Identifier.

Q11. Does it recognize every function syntax?

Answer: No. It is deliberately restricted and not a complete parser.

Q12. Why is `const vector<Token>&` used?

Answer: To avoid copying token vectors while preventing mutation through the reference.

Q13. Why `size_t`?

Answer: It is the standard unsigned type appropriate for container sizes and indexes.

Q14. Why validate ranges before matching?

Answer: To prevent empty/inverted/out-of-bounds intervals from causing incorrect access.

Q15. Why reject different lengths first?

Answer: Exact sequences cannot be equal if they contain different numbers of tokens.

Q16. What is the comparison algorithm?

Answer: Compare corresponding token types and lexemes sequentially; reject at the first difference.

Q17. What is the result of changing `+` to `-`?

Answer: No Type-1 body match because operator lexemes differ.

Q18. Does identifier renaming still match?

Answer: Not in this restricted Type-1 implementation; that is outside the exact comparison rule.

Q19. What data is in Type1BodyMatch?

Answer: Two CodeRegion records, one for each matched body.

Q20. Why separate matcher from reporter?

Answer: To separate evidence discovery from output presentation and independently test both.

Q21. Why can `std::ostringstream` be passed to an `ostream&` parameter?

Answer: `ostringstream` is an output-stream type; the reporter uses the common stream interface.

Q22. How are lines reported?

Answer: Read `line` from tokens at the begin index and at `end-1` for each region.

Q23. Why preserve newlines during comment removal?

Answer: So token line numbers still refer to the original source file.

Q24. Why use AnalyzedSourceFile?

Answer: It keeps each file path, its tokens, and its function-body regions associated in one record.

Q25. How do you avoid duplicate file pairs?

Answer: The inner loop begins at `j=i+1`, so only pairs with `i<j` are visited.

Q26. How many file pairs for 40 submissions?

Answer: 780 unique unordered file pairs; that does not prove performance at that size.

Q27. What does a candidate count mean?

Answer: The number of identical recognized body-region pairs for one file pair, not a plagiarism score.

Q28. What happens in single-file mode?

Answer: Preprocessing, tokenization, and function-region count only; no cross-file candidate report.

Q29. What tests did you run?

Answer: Seven smoke-test executables plus positive, negative, and mixed folder dataset runs.

Q30. What did the mixed dataset establish?

Answer: In one run, two pairs correctly had zero Type-1 candidates and one pair had one.

Q31. What is the biggest tokenizer limitation?

Answer: Only a supported subset of C/C++ lexical syntax and keyword vocabulary is handled.

Q32. What is the role of a PR?

Answer: It lets changes on a feature branch be reviewed before integration into `main`.

Q33. What is the supervisor’s role in similarity analysis?

Answer: Review the reported evidence in context. The tool does not determine wrongdoing.

11.4 Self-check exercises (attempt before reading answers)

Exercise 1: Given region `[4,9)`, how many tokens are included?
Answer: 5, namely indices 4 through 8.

Exercise 2: What index identifies the closing brace of `[9,14)`?
Answer: 13.

Exercise 3: Why does the inner brace region appear before the outer one?
Answer: It closes first, so its record is appended first.

Exercise 4: If three files all contain the same one-function body, how many distinct file pairs yield candidates?
Answer: 3 pairs, typically one matching body pair each.

Exercise 5: When should `printType1BodyMatches` be skipped?
Answer: When `matches.empty()` is true.

Exercise 6: Why might a `catch (...) {}` block be misclassified?
Answer: `catch` is not a recognized Keyword in this tokenizer snapshot.

Exercise 7: Does changing function name prevent a body-only match?
Answer: Not if the body token sequences remain identical.

Exercise 8: If no output appears from `git diff --cached --check`, what does it indicate?
Answer: No whitespace errors were reported for the staged diff.

12. Annotated source appendix

The following are exact source listings from the ZIP snapshot, with physical line numbers for code-review reference. The conceptual line-by-line explanations are in chapters 4–8. The appendices are included so this PDF remains useful when you are away from VS Code. The full preprocessor and tokenizer were explained separately in earlier Phase 1 materials; their salient Phase 2 dependencies are covered in chapter 3.

Complete listing: src/main.cpp

Actual source: src/main.cpp · lines 1-6 (as supplied)

```
1 | #include "application.hpp"
2 |
3 | int main()
4 | {
5 |     return runCloneScope();
6 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/application.hpp

Actual source: src/application.hpp · lines 1-5 (as supplied)

```
1 | #pragma once
2 |
3 | // Start and coordinate the CloneScope application.
4 | // Returns an exit status to the operating system.
5 | int runCloneScope();
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/code_region.hpp

Actual source: src/code_region.hpp · lines 1-15 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <cstddef>
 5 | #include <vector>
 6 | #include "tokenizer.hpp"
 7 |
 8 | struct CodeRegion
 9 | {
10 |     std::size_t beginTokenIndex;
11 |     std::size_t endTokenIndex;
12 | };
13 |
14 | std::vector<CodeRegion> findBraceRegions(
15 |     const std::vector<Token> &tokens);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/code_region.cpp

Actual source: src/code_region.cpp · lines 1-27 (as supplied)

```
 1 |
 2 | #include "code_region.hpp"
 3 |
 4 | std::vector<CodeRegion> findBraceRegions(
 5 |     const std::vector<Token> &tokens)
 6 | {
 7 |     std::vector<CodeRegion> regions;
 8 |     std::vector<std::size_t> openingBraceIndices;
 9 |
10 |     for (std::size_t i = 0; i < tokens.size(); ++i)
11 |     {
12 |         if (tokens[i].lexeme == "{")
13 |         {
14 |             openingBraceIndices.push_back(i);
15 |         }
16 |         else if (tokens[i].lexeme == "}" &&
17 |                  !openingBraceIndices.empty())
18 |         {
19 |             std::size_t start = openingBraceIndices.back();
20 |             openingBraceIndices.pop_back();
21 |
22 |             regions.push_back(CodeRegion{start, i + 1});
23 |         }
24 |     }
25 |
26 |     return regions;
```

```
27 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/function_region.hpp

Actual source: src/function_region.hpp · lines 1-8 (as supplied)

```
1 |
2 | #pragma once
3 |
4 | #include <vector>
5 | #include "code_region.hpp"
6 |
7 | std::vector<CodeRegion> findFunctionBodyRegions(
8 |     const std::vector<Token> &tokens);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/function_region.cpp

Actual source: src/function_region.cpp · lines 1-54 (as supplied)

```
1 |
2 | #include "function_region.hpp"
3 |
4 | std::vector<CodeRegion> findFunctionBodyRegions(
5 |     const std::vector<Token> &tokens)
6 | {
7 |     std::vector<CodeRegion> functionRegions;
8 |     std::vector<CodeRegion> braceRegions = findBraceRegions(tokens);
9 |
10 |     for (const CodeRegion &region : braceRegions)
11 |     {
12 |         std::size_t braceIndex = region.beginTokenIndex;
13 |
14 |         if (braceIndex == 0 || tokens[braceIndex - 1].lexeme != ")")
15 |         {
16 |             continue;
17 |         }
18 |
19 |         std::size_t depth = 0;
20 |         std::size_t openParenIndex = tokens.size();
21 |
22 |         for (std::size_t i = braceIndex; i > 0; --i)
23 |         {
24 |             std::size_t index = i - 1;
25 |
26 |             if (tokens[index].lexeme == ")")
```

```
27 |             {
28 |                 ++depth;
29 |             }
30 |             else if (tokens[index].lexeme == ")")
31 |             {
32 |                 --depth;
33 |
34 |                 if (depth == 0)
35 |                 {
36 |                     openParenIndex = index;
37 |                     break;
38 |                 }
39 |             }
40 |         }
41 |
42 |         if (openParenIndex == tokens.size() || openParenIndex == 0)
43 |         {
44 |             continue;
45 |         }
46 |
47 |         if (tokens[openParenIndex - 1].type == TokenType::Identifier)
48 |         {
49 |             functionRegions.push_back(region);
50 |         }
51 |     }
52 |
```

```
53 |     return functionRegions;
54 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/type1_clone.hpp

Actual source: src/type1_clone.hpp · lines 1-23 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <vector>
 5 | #include "code_region.hpp"
 6 |
 7 | struct Type1BodyMatch
 8 | {
 9 |     CodeRegion firstRegion;
10 |     CodeRegion secondRegion;
11 | };
12 |
13 | bool areType1BodyCandidates(
14 |     const std::vector<Token> &firstTokens,
15 |     const CodeRegion &firstRegion,
16 |     const std::vector<Token> &secondTokens,
17 |     const CodeRegion &secondRegion);
18 |
19 | std::vector<Type1BodyMatch> findType1BodyMatches(
20 |     const std::vector<Token> &firstTokens,
21 |     const std::vector<CodeRegion> &firstRegions,
22 |     const std::vector<Token> &secondTokens,
23 |     const std::vector<CodeRegion> &secondRegions);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/type1_clone.cpp

Actual source: src/type1_clone.cpp · lines 1-70 (as supplied)

```
 1 |
 2 | #include "type1_clone.hpp"
 3 |
 4 | bool areType1BodyCandidates(
 5 |     const std::vector<Token> &firstTokens,
 6 |     const CodeRegion &firstRegion,
 7 |     const std::vector<Token> &secondTokens,
 8 |     const CodeRegion &secondRegion)
 9 | {
10 |     if (firstRegion.beginTokenIndex >= firstRegion.endTokenIndex ||
11 |         secondRegion.beginTokenIndex >= secondRegion.endTokenIndex ||
12 |         firstRegion.endTokenIndex > firstTokens.size() ||
13 |         secondRegion.endTokenIndex > secondTokens.size())
14 |     {
15 |         return false;
16 |     }
17 |
18 |     std::size_t firstLength =
19 |         firstRegion.endTokenIndex - firstRegion.beginTokenIndex;
20 |
21 |     std::size_t secondLength =
22 |         secondRegion.endTokenIndex - secondRegion.beginTokenIndex;
23 |
24 |     if (firstLength != secondLength)
25 |     {
26 |         return false;
```

```
27 |     }
28 |
29 |     for (std::size_t i = 0; i < firstLength; ++i)
30 |     {
31 |         const Token &first =
32 |             firstTokens[firstRegion.beginTokenIndex + i];
33 |
34 |         const Token &second =
35 |             secondTokens[secondRegion.beginTokenIndex + i];
36 |
37 |         if (first.type != second.type ||
38 |             first.lexeme != second.lexeme)
39 |         {
40 |             return false;
41 |         }
42 |     }
43 |
44 |     return true;
45 | }
46 |
47 | std::vector<Type1BodyMatch> findType1BodyMatches(
48 |     const std::vector<Token> &firstTokens,
49 |     const std::vector<CodeRegion> &firstRegions,
50 |     const std::vector<Token> &secondTokens,
51 |     const std::vector<CodeRegion> &secondRegions)
52 | {
```

```
53 |     std::vector<Type1BodyMatch> matches;
54 |
55 |     for (const CodeRegion &firstRegion : firstRegions)
56 |     {
57 |         for (const CodeRegion &secondRegion : secondRegions)
58 |         {
59 |             if (areType1BodyCandidates(
60 |                     firstTokens, firstRegion,
61 |                     secondTokens, secondRegion))
62 |             {
63 |                 matches.push_back(
64 |                     Type1BodyMatch{firstRegion, secondRegion});
65 |             }
66 |         }
67 |     }
68 |
69 |     return matches;
70 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/type1_report.hpp

Actual source: src/type1_report.hpp · lines 1-16 (as supplied)

```
 1 |
 2 | #pragma once
 3 |
 4 | #include <ostream>
 5 | #include <string>
 6 | #include <vector>
 7 |
 8 | #include "type1_clone.hpp"
 9 |
10 | void printType1BodyMatches(
11 |     std::ostream &output,
12 |     const std::string &firstFileName,
13 |     const std::vector<Token> &firstTokens,
14 |     const std::string &secondFileName,
15 |     const std::vector<Token> &secondTokens,
16 |     const std::vector<Type1BodyMatch> &matches);
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/type1_report.cpp

Actual source: src/type1_report.cpp · lines 1-57 (as supplied)

```
 1 |
 2 | #include "type1_report.hpp"
 3 |
 4 | void printType1BodyMatches(
 5 |     std::ostream &output,
 6 |     const std::string &firstFileName,
 7 |     const std::vector<Token> &firstTokens,
 8 |     const std::string &secondFileName,
 9 |     const std::vector<Token> &secondTokens,
10 |     const std::vector<Type1BodyMatch> &matches)
11 | {
12 |     output << "CloneScope Type-1 body-level candidate report\n";
13 |
14 |     std::size_t reportedCount = 0;
15 |
16 |     for (const Type1BodyMatch &match : matches)
17 |     {
18 |         const CodeRegion &first = match.firstRegion;
19 |         const CodeRegion &second = match.secondRegion;
20 |
21 |         if (first.beginTokenIndex >= first.endTokenIndex ||
22 |             first.endTokenIndex > firstTokens.size() ||
23 |             second.beginTokenIndex >= second.endTokenIndex ||
24 |             second.endTokenIndex > secondTokens.size())
25 |         {
26 |             output << "Warning: skipped invalid match boundaries.\n";
```

```
27 |             continue;
28 |         }
29 |
30 |         const Token &firstStart =
31 |             firstTokens[first.beginTokenIndex];
32 |
33 |         const Token &firstEnd =
34 |             firstTokens[first.endTokenIndex - 1];
35 |
36 |         const Token &secondStart =
37 |             secondTokens[second.beginTokenIndex];
38 |
39 |         const Token &secondEnd =
40 |             secondTokens[second.endTokenIndex - 1];
41 |
42 |         ++reportedCount;
43 |
44 |         output << "\nCandidate #" << reportedCount << "\n";
45 |
46 |         output << firstFileName << ": lines "
47 |                << firstStart.line << " to " << firstEnd.line << "\n";
48 |
49 |         output << secondFileName << ": lines "
50 |                << secondStart.line << " to " << secondEnd.line << "\n";
51 |
52 |         output << "Evidence: identical body token types and lexemes.\n";
```

```
53 |         output << "Human review required; similarity is not proof of plagiarism.\n";
54 |     }
55 |
56 |     output << "\nReported candidates: " << reportedCount << "\n";
57 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Complete listing: src/application.cpp

Actual source: src/application.cpp · lines 1-197 (as supplied)

```
 1 |
 2 | #include <iostream>
 3 |
 4 | #include <fstream>
 5 |
 6 | #include <string>
 7 |
 8 | #include <filesystem>
 9 |
10 | #include <sstream>
11 |
12 | #include "application.hpp"
13 |
14 | #include "preprocessor.hpp"
15 |
16 | #include "tokenizer.hpp"
17 |
18 | #include "function_region.hpp"
19 |
20 | #include "type1_clone.hpp"
21 |
22 | #include "type1_report.hpp"
23 |
24 | #include <vector>
25 |
26 | using namespace std;
```

```
27 |
28 | namespace fs = std::filesystem;
29 |
30 | struct AnalyzedSourceFile
31 | {
32 |     fs::path path;
33 |     std::vector<Token> tokens;
34 |     std::vector<CodeRegion> functionRegions;
35 | };
36 |
37 | int runCloneScope()
38 | {
39 |     cout << "================================\n";
40 |     cout << "        CloneScope v0.1\n";
41 |     cout << "================================\n";
42 |     cout << "Code Clone Analysis Tool\n";
43 |     cout << "Status: Development Started\n";
44 |
45 |     // Read a user-selected C/C++ source file
46 |     // Ask the user for a source file path
47 |     string filePath;
48 |
49 |     cout << "Enter source file path: ";
50 |     getline(cin, filePath);
51 |
52 |     fs::path sourcePath(filePath);
```

```
    // Check whether the input path is a directory
    if (fs::is_directory(sourcePath))
    {
        cout << "Folder detected successfully!\n";

        std::vector<AnalyzedSourceFile> analyzedFiles;

        // Visit each entry directly inside the folder
        for (const auto &entry : fs::directory_iterator(sourcePath))
        {
            if (entry.is_regular_file() &&
                (entry.path().extension() == ".c" ||
                 entry.path().extension() == ".cpp"))
            {
                cout << entry.path() << '\n';
                // Open the discovered source file
                ifstream sourceFile(entry.path());

                // Handle a file-opening failure
                if (!sourceFile.is_open())
                {
                    cerr << "Error: Could not open " << entry.path() << '\n';
                    continue;
                }

                cout << "Source file opened successfully!\n";

                // Read the complete discovered source file
                std::ostringstream buffer;
                buffer << sourceFile.rdbuf();

                std::string sourceCode = buffer.str();

                // Remove comments from this source file
                std::string cleanedCode = removeComments(sourceCode);

                // Display the processed source code
                cout << "Preprocessed source code:\n";
                cout << cleanedCode << '\n';

                // Tokenize the current source file.
                std::vector<Token> tokens = tokenize(cleanedCode);

                // Display the total token count for this file.
                cout << "Total tokens: " << tokens.size() << '\n';

                std::vector<CodeRegion> functionRegions =
                    findFunctionBodyRegions(tokens);

                cout << "Supported function-body regions: "
                     << functionRegions.size() << '\n';
```

```
105 |
106 |                 analyzedFiles.push_back(
107 |                     AnalyzedSourceFile{entry.path(), tokens, functionRegions});
108 |             }
109 |         }
110 |
111 |         cout << "\nAnalyzed source files stored: "
112 |              << analyzedFiles.size() << '\n';
113 |
114 |         std::size_t filePairCount = 0;
115 |
116 |         for (std::size_t i = 0; i < analyzedFiles.size(); ++i)
117 |         {
118 |             for (std::size_t j = i + 1; j < analyzedFiles.size(); ++j)
119 |             {
120 |                 const AnalyzedSourceFile &first = analyzedFiles[i];
121 |                 const AnalyzedSourceFile &second = analyzedFiles[j];
122 |
123 |                 cout << "Comparing files: "
124 |                      << first.path.filename()
125 |                      << " vs "
126 |                      << second.path.filename() << '\n';
127 |
128 |                 std::vector<Type1BodyMatch> matches =
129 |                     findType1BodyMatches(
130 |                         first.tokens,
```

```
131 |                         first.functionRegions,
132 |                         second.tokens,
133 |                         second.functionRegions);
134 |
135 |                 cout << "Type-1 body-level candidates: "
136 |                      << matches.size() << '\n';
137 |
138 |                 if (!matches.empty())
139 |                 {
140 |                     printType1BodyMatches(
141 |                         cout,
142 |                         first.path.filename().string(),
143 |                         first.tokens,
144 |                         second.path.filename().string(),
145 |                         second.tokens,
146 |                         matches);
147 |                 }
148 |
149 |                 ++filePairCount;
150 |             }
151 |         }
152 |
153 |         cout << "Total source-file pairs: "
154 |              << filePairCount << '\n';
155 |
156 |         return 0;
```

```
157 |     }
158 |
159 |     // Open the selected source file
160 |     ifstream inputFile(sourcePath);
161 |
162 |     // Check whether the file opened successfully
163 |     if (!inputFile.is_open())
164 |     {
165 |         cerr << "Error: Could not open source file.\n";
166 |         return 1;
167 |     }
168 |
169 |     cout << "Source file opened successfully!\n";
170 |
171 |     // Read the complete source file into memory
172 |     std::ostringstream buffer;
173 |     buffer << inputFile.rdbuf();
174 |
175 |     std::string sourceCode = buffer.str();
176 |
177 |     // Remove comments from the source code
178 |     std::string cleanedCode = removeComments(sourceCode);
179 |
180 |     // Display the preprocessed source code.
181 |     cout << "Preprocessed source code:\n";
182 |     cout << cleanedCode;
```

```
183 |
184 |     // Convert the preprocessed source into tokens.
185 |     std::vector<Token> tokens = tokenize(cleanedCode);
186 |
187 |     // Display how many tokens were produced.
188 |     cout << "\nTotal tokens: " << tokens.size() << '\n';
189 |
190 |     std::vector<CodeRegion> functionRegions =
191 |         findFunctionBodyRegions(tokens);
192 |
193 |     cout << "Supported function-body regions: "
194 |          << functionRegions.size() << '\n';
195 |
196 |     return 0;
197 | }
```

Line numbers above refer to the uploaded source snapshot; whitespace-only lines are intentionally retained.

Appendix: test and file inventory

`tests/code_region_smoke.cpp`

`tests/function_region_smoke.cpp`

`tests/pipeline_smoke.cpp`

`tests/preprocessor_smoke.cpp`

`tests/tokenizer_smoke.cpp`

`tests/type1_clone_smoke.cpp`

`tests/type1_file_pipeline_smoke.cpp`

Fixture groups: `tests/fixtures/type1/*.cpp`, `tests/fixtures/type1/negative/*.cpp`, and `tests/fixtures/type1/mixed/*.cpp`. For full test assertions refer to the supplied ZIP and your repository. The missing `samples/hello.c` is a packaging gap in this uploaded bundle, not a claim that the original test never passed.

Final supervisor-ready summary

One-sentence explanation: CloneScope currently compares selected function bodies across multiple C/C++ student submissions using exact token matching and shows instructors where matching code occurs, without assigning plagiarism guilt.

Technical explanation: It applies a five-state comment remover, a restricted lexer with source positions, stack-style brace interval extraction, heuristic function-body identification, pairwise exact token-type-plus-lexeme comparison, and stream-based source-line reporting, wrapped in a non-recursive folder-processing C++17 application.

> **END OF PHASE 2 LEARNING MANUAL**
>
> You should now be able to trace a sample from raw characters to candidate report, justify every major C++ data structure and loop, reproduce the test results, and explain precisely why the tool reports *candidates for human review* rather than plagiarism decisions.